

VGA-lab

TSEA83 Datorkonstruktion

Anders Nilsson

2024 version 3.1

Innehåll

1	Introduktion	3
1.1	Syfte	3
1.2	Förkunskaper	3
1.3	Material	3
1.4	Förberedelser	3
2	Blockschemakonstruktion	4
2.1	Förutsättningar	4
2.2	Översikt	4
2.3	KBD_ENC : Tangentbordsavkodaren	4
2.4	PRET_CPU : "Låtsas-datorn"	6
2.5	VIDEO_RAM : Bildminnet	6
2.6	VGA_MOTOR : Bildgeneratorn	7
2.6.1	CHAR_ROM : Teckenminnet	7
3	Uppgifter	10
3.1	Uppgift 1 : Skapa bild	10
3.2	Uppgift 2 : Läs av tangentbordet	11
3.3	Extrauppgift : Skala om	11
4	Bilagor	13
4.1	PS/2 scan codes	13
4.2	VGA timing	13
5	Versionslog	14

1 Introduktion

1.1 Syfte

Den här laborationen kommer att beröra flera saker. Avkodning av signaler från ett PS/2-tangentbord. Användning av ett block-RAM som bildminne. Generering av VGA-signaler till en bildskärm. Dessa tre delar sätts samman i en större hierakisk konstruktion som bildar en simpel textmonitor där man kan skriva och redigera text. Alla delar kan vara nyttiga i det kommande datorkonstruktionsprojektet.

1.2 Förkunskaper

För att på ett bra sätt tillgodogöra sig laborationen behöver man ha grunderna från tidigare digitalteknikkurs. Man bör också ha gjort en föregående laboration i VHDL för att vara något bekant med språket och labmiljön.

1.3 Material

Allt material som behövs till laborationen finns i labsalen, även labinstruktionen (denna skrift), fast bara då i elektronisk form som PDF-fil från kursen websida. Vill man ha en tryckt kopia så får man skriva ut den själv. Detta är dock inte nödvändigt för att kunna utföra själva laborationen.

1.4 Förberedelser

Läs igenom följande avsnitt, Blockschemakonstruktion. Studera den givna VHDL-koden som kan laddas ned från kursens web-sida, och jämför blockschema och kod så att du förstår vad som händer. Förbered uppgifterna i avsnittet Uppgifter så att du kan använda labtiden till att prova och eventuellt felsöka i koden.

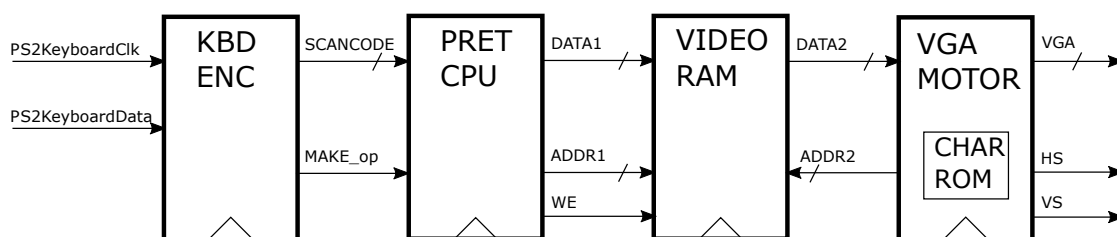
2 Blockschemakonstruktion

2.1 Förutsättningar

Textmonitorn ska visas på en VGA-skärm i upplösningen 640×480 pixlar. Detta uppnås med korrekt VGA-timing (se bilaga 4.2). Textmonitorn är konstruerad till att ha 15 rader á 20 kolumner.

2.2 Översikt

Hela konstruktionen av textmonitorn framgår av figur 2.1.



Figur 2.1: Textmonitor

Tangentbordsavkodaren (KBD_ENC) läser av tangentbordets PS/2-signaler och producerar en scancode till "låtsas-datorn" (PRET_CPU) som skriver in lämpliga data i bildminnet (VIDEO_RAM). Samtidigt läser bildgeneratoren (VGA_MOTOR) ur bildminnet och skapar lämpliga synk- och VGA-signaler till en VGA-monitor.

I den följande framställningen kommer vi stegvis att dela upp konstruktionen i mindre realiserbara delar för att lättare kunna överföra dessa till fungerande VHDL-kod.

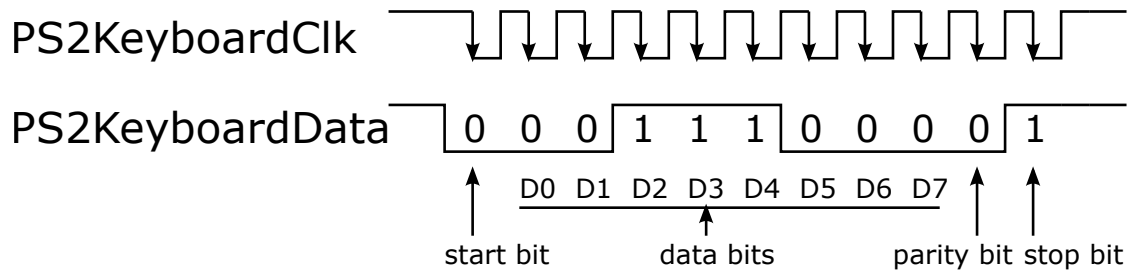
2.3 KBD_ENC : Tangentbordsavkodaren

Tangentbordsavkodarens uppgift är att läsa av PS/2-signaler från tangentbordet, och skapa en scancode med de tillhörande signalerna make och break.

När man trycker ned en tangent på ett tangentbord så skickas data om tangettryckningen seriellt över två ledningar, PS2KeyboardClk och PS2KeyboardData. Data som skickas utgörs av scankoder (se appendix 4.1). Själva nedtryckningen av tangenten kallas för MAKE, och då skickas tangentens scankod. När man släpper upp tangenten kallas detta för BREAK, och då skickas koden $\$F0$ följt av scankoden för den aktuella tangenten. För t ex tangenten A skickas scankoden $\$1C$ vid MAKE, och $\$F0$ följt av $\$1C$ vid BREAK. Se

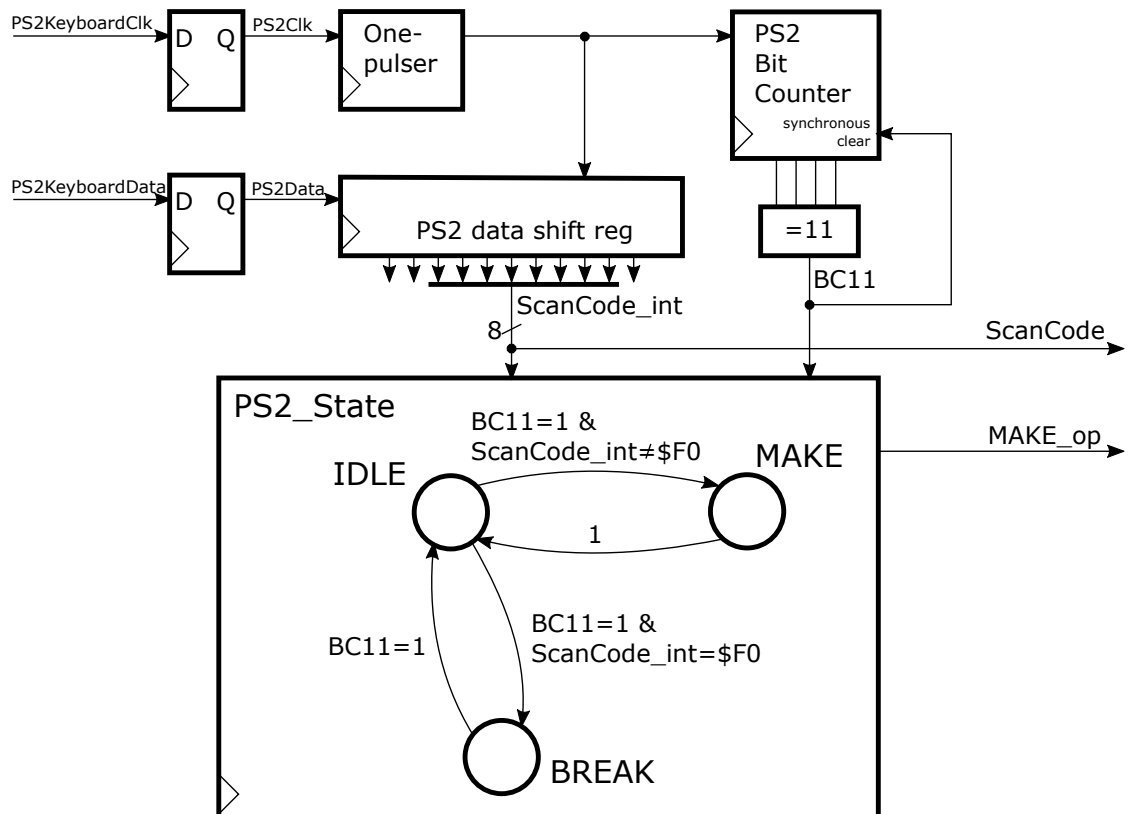
exempel i figur 2.2. Observera att avläsningen av varje bit sker vid den negativa flanken på PS2KeyboardClk.

Vissa tangenter har utökade scankoder, dvs det skickas mer än en byte vid MAKE. I vår lösning kommer vi att bortse från det och endast hantera enstaka byte vid MAKE.



Figur 2.2: PS2, scankod för tangenten A

Lösningen av tangentbordsavkodaren behöver delas upp ytterligare innan vi kan göra VHDL-kod av det. Vi gör det enligt figur 2.3.



Figur 2.3: KBD_ENC

Tangentbordets signaler synkroniseras med vippor. PS2-klockans negativa flank enpulsas och skiftar in databitarna i ett skiftregister samt räknar alla 11 bitarna i en bit-räknare. Själva scankoden och signalen BC11 som säger att alla 11 bitar tagits emot går in i en tillståndsmaskin (PS2_State) som avgör om MAKE eller BREAK har inträffat. Allt detta kan direkt översättas till VHDL.

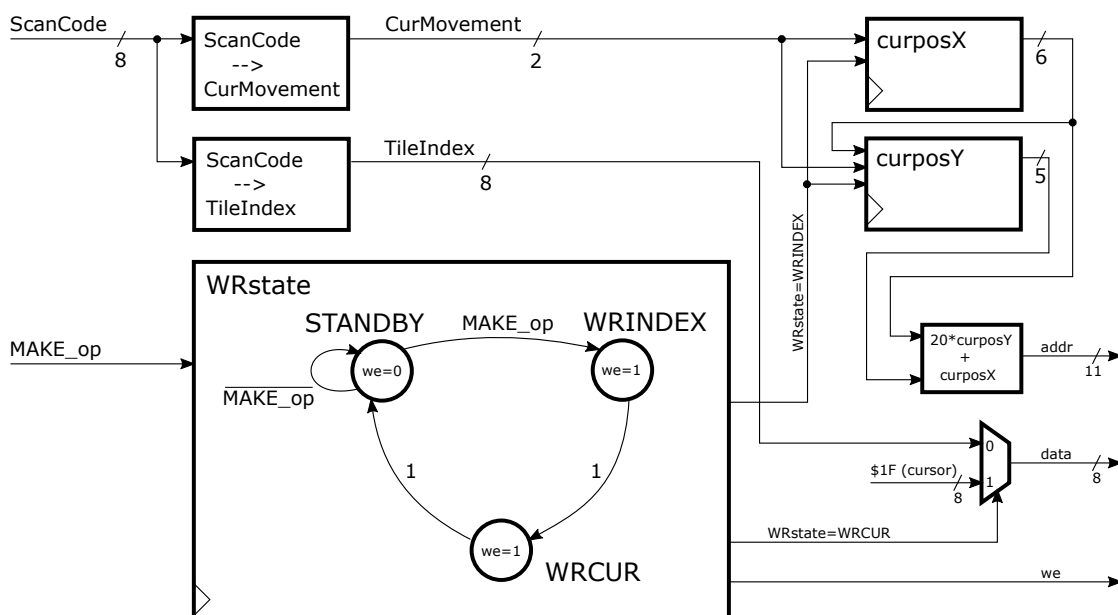
2.4 PRET_CPU : "Låtsas-datorn"

Det andra blocket, ”låtsas-datorn” (PRET_CPU), kommer här att ha en hårdkodad funktionalitet. Dvs, den är inte programmerbar. Funktionaliteten av PRET_CPU behöver också förtydligas. Vi gör det i figur 2.4.

Scankoden översätts i två tabeller. Dels till något som talar om hur cursorn ska röra sig (framåt, bakåt eller ny rad). Skriver man in en bokstav så ska cursorn röra sig framåt, vid backspace ska den gå bakåt och för enter-tangenten ska den gå till en ny rad. Dessutom översätts scankoden till ett uppräkningsbart index som lämpar sig bättre att skriva in i bildminnet än själva scankoden.

Var gång man trycker på en tangent så ska två skrivningar göras i bildminnet. Detta sker i tillståndsmaskinen `WRState`. Först ska cursorns nuvarande position skrivas över av den nya tangentens index, och därefter ska själva cursorns index skrivas på cursorns nya position.

Cursors position sköts av processerna `curposX` och `curposY`. Dessa bildar tillsammans adressen till bildminnet via en enkel multiplikation och addition. Alla delar kan mer eller mindre direkt översättas till VHDL.

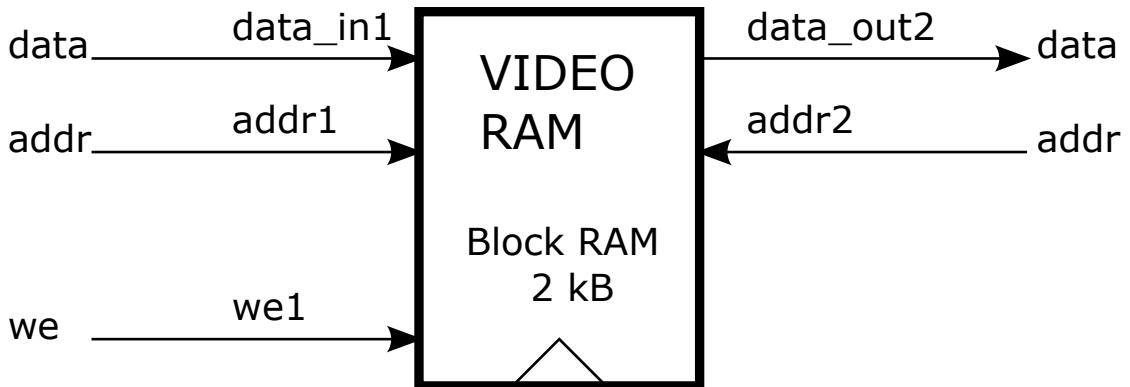


Figur 2.4: PRET_CPU

2.5 VIDEO_RAM : Bildminnet

Bildminnet (VIDEO_RAM) består av ett 2kB stort block-RAM. Då textmonitorn består av 15 rader á 20 kolumner med ett tecken i varje position så behövs endast 300 byte, men minsta allokerbara block-RAM-storlek i vår FPGA är just 2kB, så det får bli det. Block-RAM:et är ett dubbelports-minne där tangentbordsavkodaren kan skriva in ny information på en adress via ena porten samtidigt som bildgeneratoren kan läsa ur minnet

på en annan (eller samma) adress via den andra porten. Just denna egenskap gör att block-RAM:et lämpar sig mycket väl som bildminne. En block-RAMs-konstruktion kan direkt skrivas i VHDL.



Figur 2.5: Bildminnet VIDEO_RAM

2.6 VGA_MOTOR : Bildgeneratorm

Slutligen då bildgeneratorm (VGA_MOTOR). Dess uppgift är att läsa indexvärden ur bildminnet och rita ut motsvarande teckens utseende på bildskärmen. Varje teckens utseende finns i bildgeneratorms teckenminne (CHAR_ROM). För att VGA-monitorm ska visa en korrekt bild måste synksignaler (Vsync och Hsync) samt en släcksignal (Blank) genereras. Allt i bildgeneratorm har sitt ursprung från två pixelräknare (Xpixel och Ypixel) vilka styr hela funktionen. För att nå den tänkta bildskärmsupplösningen på 640×480 pixlar behöver dessa räknare gå i en takt av 25 MHz. Denna frekvens genereras av en modulo-4-räknare där utsignalen ligger hög under endast en klockcykels längd av den ursprungliga 100 MHz-klockan.

2.6.1 CHAR_ROM : Teckenminnet

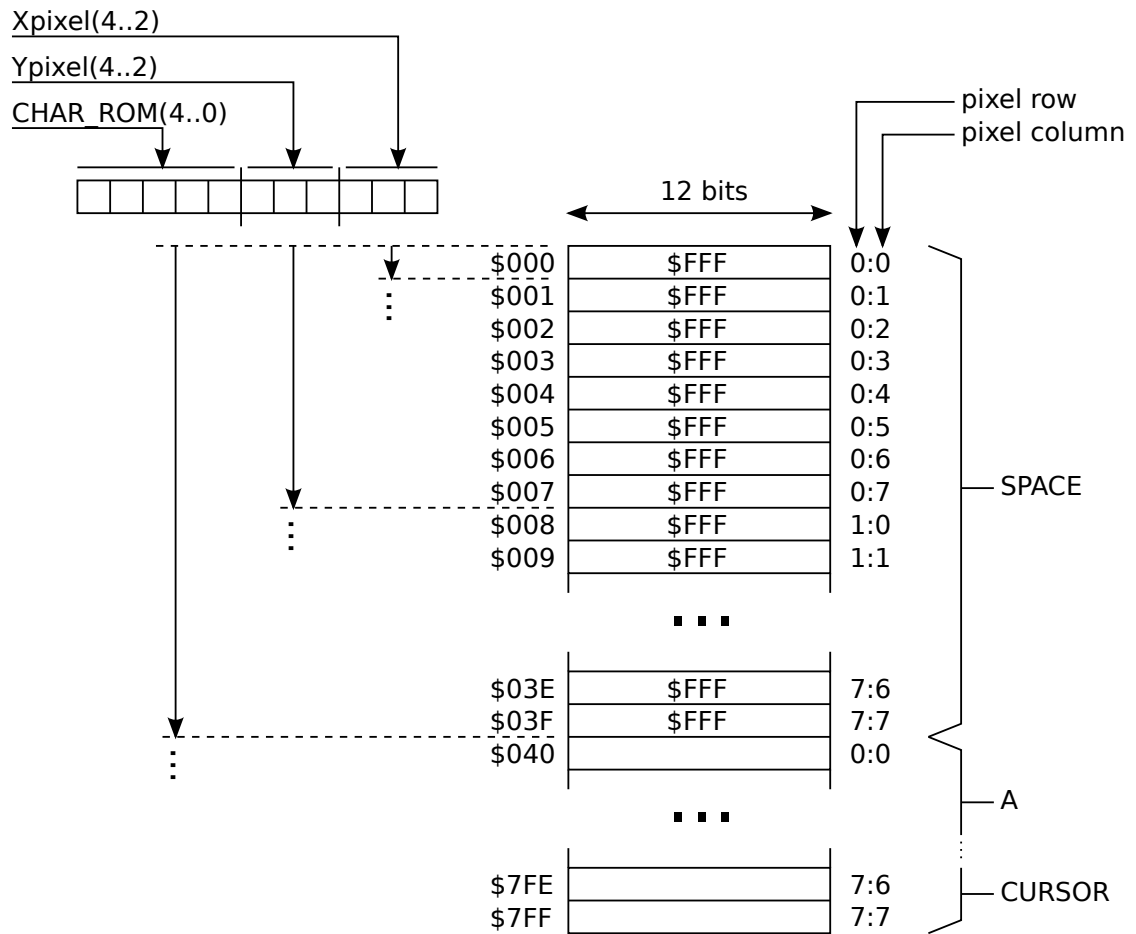
Teckenminnet (CHAR_ROM) ska innehålla utseendet för alla tecken som vi vill kunna visa. Det går att göra detta på många tänkbara sätt beroende på vad man vill åstadkomma. Vi vill att varje pixel i ett tecken ska kunna anta alla tillgängliga färger, dvs varje pixel kommer att motsvaras av ett 12-bitars-värde, där \$FFF är helt vitt och \$000 är helt svart. Värden däremellan kommer att ge olika färger beroende på dom ingående bitarna för dom röda, gröna och blåa komponenterna i 12-bitars-ordet.

Teckeminnet organiseras alltså på så sätt att varje adress innehar färgvärdet för en pixel. Vi vill också att varje tecken har en storlek av 8×8 pixlar. Det medför att varje tecken upptar totalt 64 adresser á 12 bitar. De 64 första adresserna av teckenminnet innehåller alla värdet \$FFF och motsvarar alltså utseendet för SPACE (mellanslag). Därefter följer utseendet för alfabetets olika tecken A, B, C och så vidare till Å, Ä och Ö. Slutligen på dom 64 sista adresserna i minnet finns utseendet för själva cursorn.

Adresseringen av teckenminnet är organiserat så att varje pixel faktiskt läses inte mindre än 16 gånger, men vid olika tillfällen. Via genomsvepningen av bildskärmens upplösning

Lodräta streckade röda linjer indikerar var signaler befinner sig vid nästkommande klockflank.

8



Figur 2.7: Teckenminnet CHAR_ROM

3 Uppgifter

Hela hårdvarukonstruktionen är given enligt föregående avsnitt, Blockschemakonstruktion. Stora delar av VHDL-koden är också given. Koden finns att tanka ned från kursens hemsida för den här labben. Din uppgift är att fylla i VHDL-kod där funktionalitet saknas. Detta görs lämpligen i en viss ordning enligt följande avsnitt.

Tanka ned lab-skelettet `vga_lab.tgz` och packa upp med kommandot:

```
tar xzvf vga_lab.tgz
```

Ladda även modulen TSEA83 med kommandot:

```
module add courses/TSEA83
```

3.1 Uppgift 1 : Skapa bild

Gå till underkatalogen `VGA_MOTOR` och gör hela uppgift 1 därifrån.

Bildgeneratoren (`VGA_MOTOR`) saknar viss funktionalitet för att ge korrekta synksignaler. Komplettera koden i `VGA_MOTOR.vhd`, dvs blocken `Xpixel`, `Ypixel`, `Vsync`, `Hsync` och `Blank`.

Simulera `VGA_MOTOR` med kommandot:

```
make sim
```

Om du kör simuleringen i 20 ms bör du få ett resultat enligt figur 3.1. Kontrollera att det kommer en `vsync` och massor med `hsync`. Det går även att mäta och kontrollera avstånd mellan och längd på `hsync`-pulserna. Simulerar du i ytterligare 20 ms kan du även kontrollera avståndet mellan `vsync`-pulserna.

Om simuleringen inte alls liknar detta kan det vara aktuellt att lägga till signalerna för `Xpixel`- och `Ypixel`-räknarna i undermodulen `U2` (`vga_motorn`), och kontrollera att dessa räknar som dom ska.

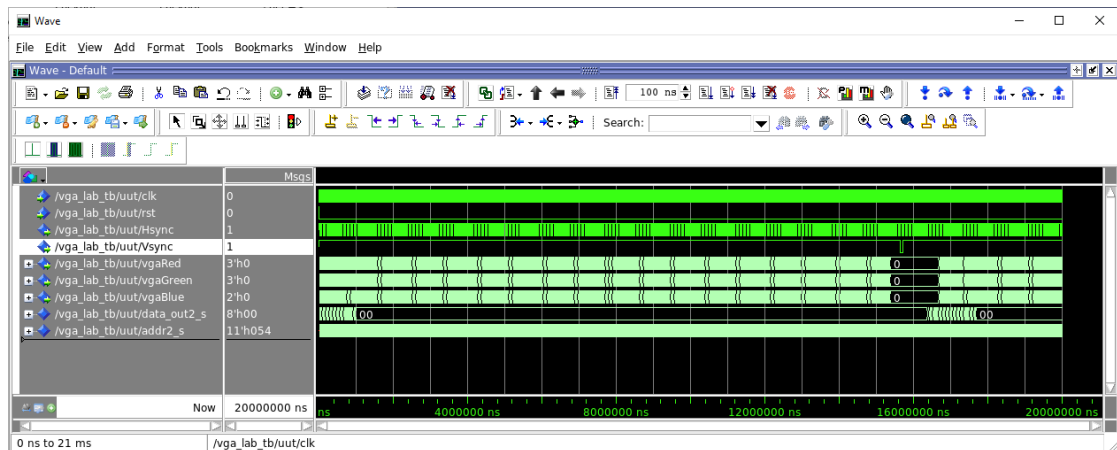
Syntetisera sedan till en bit-fil med kommandot:

```
make bit
```

Ladda in bit-filen i Basys3-kortet med kommandot:

```
make prog
```

Om du gjort allt rätt ska du se en vit bakgrund med en pacman-liknande markör uppe i vänstra hörnet. Det kan tänkas att du behöver trycka på reset (mittenknappen - `btnC`) så att alla delar av konstruktionen är i synk och har rätt initialvärde.



Figur 3.1: Simulering av VGA_MOTOR

3.2 Uppgift 2 : Läs av tangentbordet

Gå nu upp en nivå till katalogen VGA_lab och gör hela uppgift 2 därifrån.

Tangentbordsavkodaren (KBD_ENC) saknar viss funktionalitet för att generera en ScanCode samt Make och Break. Komplettera koden för blocken PS2BitCounter, PS2DataShiftReg, PS2state och One-pulser i filen KBD_ENC.vhd.

Simulera VGA_lab med kommandot:

```
make sim
```

Om du kör simuleringen i 20 ms, och lägger till signalerna WRstate från undermodulen U1 och signalen VideoRAM från undermodulen U2, bör du få ett resultat enligt figur 3.2. Signalerna PS2KeyboardC1k och PS2KeyboardData simulerar Make och Break för tangenttryckningar av A, B och Y i den ordningen. Om man zoomar in på signalen WRstate ska man kunna se hur tillståndet cyklar mellan STANDBY, WRINDEX och WRCUR för varje tangenttryckning. Expanderar man innehållet i signalen VideoRAM ska man kunna se hur index 01, 02 och 19 (för tecknen A, B respektive Y) skrivs in i bildminnet. Index 1F är index för tecknet som motsvarar pacman-figuren (dvs cursorn).

Syntetisera sedan till en bit-fil med kommandot:

```
make bit
```

Ladda in bit-filen i Basys3-kortet med kommandot:

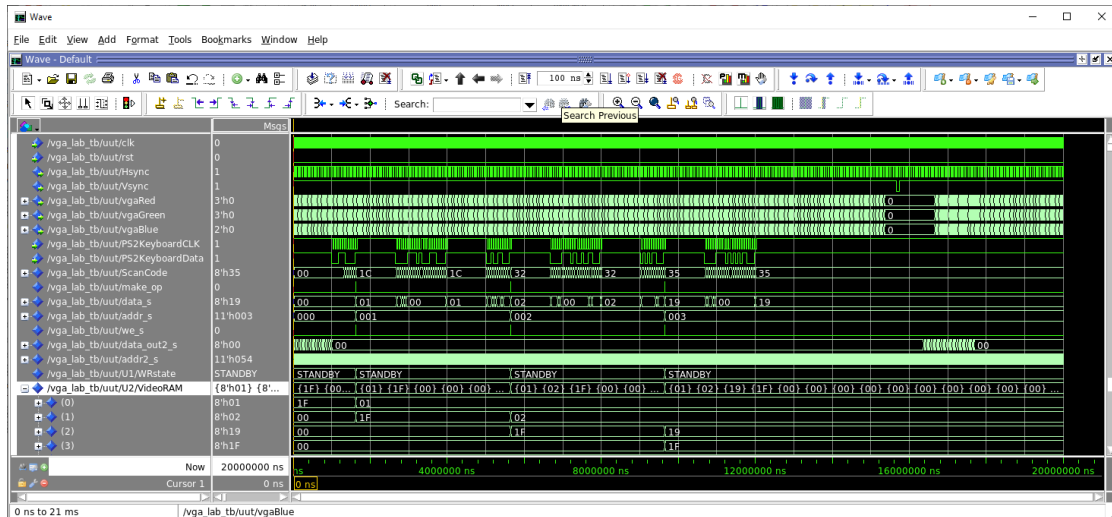
```
make prog
```

Om du gjort rätt ska du kunna skriva text (dock bara bokstäverna A, B, C, X, Y, Z, Å, Ä, Ö samt SPACE) i textmonitorn via det externa tangentbordet inkopplat till Basys3-kortet. Kom ihåg reset.

3.3 Extrauppgift : Skala om

Gör extrauppgiften i mån av tid. Du behöver inte göra den för att vara godkänd på labben.

3 Uppgifter



Figur 3.2: Simulering av VGA_lab

Textmonitorn är gjord för att visa 15 rader á 20 kolumner. Skala om visningen till det dubbla, dvs 30 rader á 40 kolumner genom att ändra i VHDL-koden på lämpliga ställen.

Kompletera även teckenminnet (CHAR_ROM) med utseende på odefinierade bokstäver så att du t ex kan skriva ditt namn.

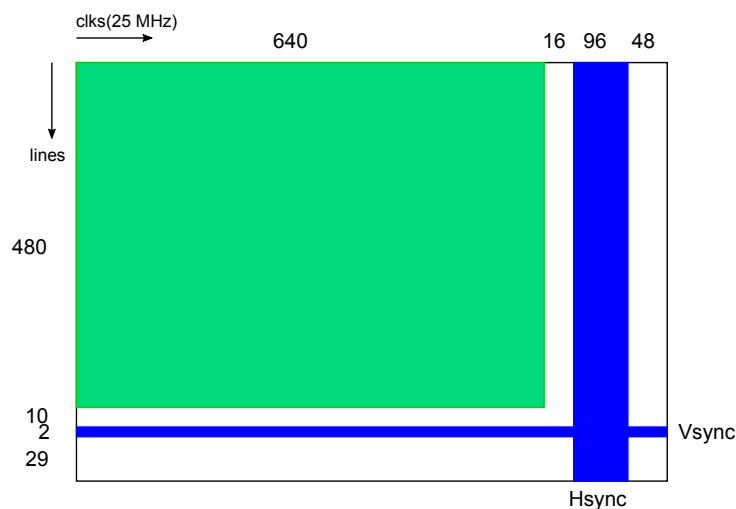
4 Bilagor

4.1 PS/2 scan codes

ESC 76	F1 05	F2 06	F3 04	F4 0C	F5 03	F6 0B	F7 83	F8 0A	F9 01	F10 09	F11 78	F12 07	↑ E0 75
~ 0E	1! 16	2@ 1E	3# 26	4\$ 25	5% 2E	6^ 36	7& 3D	8* 3E	9(46	0) 45	-+ 4E	BackSpace ← 66	→ E0 74
TAB 0D	Q 15	W 1D	E 24	R 2D	T 2C	Y 35	U 3C	I 43	O 44	P 4D	[{ 54	]} 5B	↵ E0 6B
Caps Lock 58	A 1C	S 1B	D 23	F 2B	G 34	H 33	J 3B	K 42	L 4B	:: 4C	"" 52	Enter ↵ 5A	↓ E0 72
Shift 12	Z 1Z	X 22	C 21	V 2A	B 32	N 31	M 3A	,< 41	>. 49	/ ? 4A	⇧ 59	Shift 59	
Ctrl 14	Alt 11	Space 29							Alt E0 11	Ctrl E0 14			

PS/2 Keyboard Scan Codes

4.2 VGA timing



Figur 4.1: VGA timing. Vsync och Hsync är aktivt låga.

Utöver den synliga delen av bilden (640×480 pixlar grön yta) behöver synksignaler (Vsync och Hsync, blå yta) skapas under vissa tider för att VGA-monitorn ska kunna visa bilden. Utanför den synliga delen av bilden (dvs blå och vit yta) måste videosignalen vara svart, dvs s k blanking ska aktiveras där.

5 Versionslog

2016 version 1.0	Ursprunglig version
2017 version 1.1	Diverse omritade figurer
2020 version 1.2	Figur 2.6, VGA_MOTOR, förtydligande av CLK-signal
2021 version 1.3	Simuleringsinformation med figurer och beskrivning för uppgift 1 och 2.
2023 version 2.0	Omstrukturering: KBD_ENC uppdelad i KBD_ENC och PRET_CPU PICT_MEM byter namn till VIDEO_RAM TILE_MEM byter namn till CHAR_ROM VGA_MOTOR pipelineas korrekt
2023 version 2.1	Figur 2.1, 2.3 och 2.4 ändrade m a p signalen MAKE_op
2024 version 3.0	Anpassning till Basys3 12-bitars VGA (påverkar text och bild 2.6 och 2.7) Nytt make-system (påverkar uppgiftsbeskrivning, avsnitt 3)
2024 version 3.1	Påminnelse om att trycka på reset efter prog